The following code is assumed to count the internal

The following code is assumed to count the internal

The following code is assumed to count the internal nodes with 2 children of an incomplete binary tree. Choose the most specific answer from below:

```
count_2_children(t(L,_,R),N):-var(L),!,
     count_2_children(R,N).
count_2_children(t(L,_,R),N):-var(R),!,
     count_2_children(L,N).
count_2_children(t(L,_,R),NN):-
     count_2_children(R,NR),
     count_2_children(L,NL),
     NN is NL+NR+1.
count_2_children(t(L,_,R),0):-
     var(L),var(R),!.
```

Select one:

- a. The predicate is incorrect as the last clause should be the first one.
- b. The predicate is incorrect as the clause right subtree is processed before the left one.
- c. The predicate is correct.
- d. The predicate is incorrect, as is missing a clause on `var(T).`

The following sequence is assumed to be used in a condition-

Question 3
Not yet answered
Marked out of 1.00
Flag question

If we want a **decreasing** sorting by selection strategy (below), where `max` and `delete` are correct predicates to find the max value, and delete a value from a list respectively, which condition must hold true (choose the most specific one):

```
sort(In,[M|Out]):-
     max(In,M),
     delete(M,In,Int),
     sort(Int,Out).
sort([],[]).
```

Select one:

- a. `min` needs to use a strict inequality
- b. `delete` needs to delete the first occurrence of the value from the list
- c. `max` needs to choose the first occurrence of the largest value from the list
- d. `min` needs to be implemented with forward recursion

The following sequence is assumed to be used in a condition….

The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort. Would it be a sound choice (that is, can be bubble sort implemented using it)?

```
traverse_once([A,B|T],[B|R]):-
      A>=B,!,
      traverse_once ([A|T],R).
traverse_once([A|T],R):-
      traverse_once(T,R).
traverse_once([],[]).
```

Select one:

- a. No! It (traverse_once) fails as it cannot reach the clause with the empty list due to patterns in the other clauses.
- b. No! A bubble sort using it might enter an infinite loop in case duplicates exist in the input.
- c. No! It (traverse_once) fails as fact should come as first clause.
- d. Yes! If properly employed, that is, until no swap occurred in the latest traverse_once.

If we want a decreasing sorting by selection …

It we want a decreasing sorting by selection …

Question **3**
Not yet answered
Marked out of 1.00
Flag question

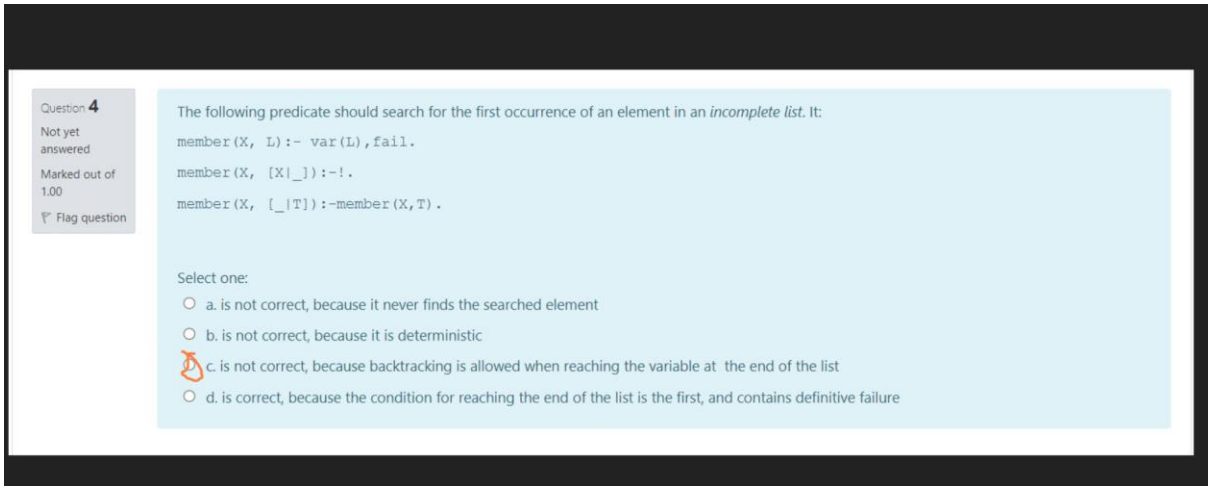
If we want a **decreasing** sorting by selection strategy (below), where `max` and `delete` are correct predicates to find the max value, and delete a value from a list respectively, which condition must hold true (choose the most specific one):

```
sort(In,[M|Out]):-
      max(In,M),
      delete(M,In,Int),
      sort(Int,Out).
sort([],[]).
```

Select one:

- a. `min` needs to use a strict inequality
- b. `delete` needs to delete the first occurrence of the value from the list
- c. `max` needs to choose the first occurrence of the largest value from the list
- d. `min` needs to be implemented with forward recursion

The following predicate should search …



Question 4

Not yet answered

Marked out of 1.00

Flag question

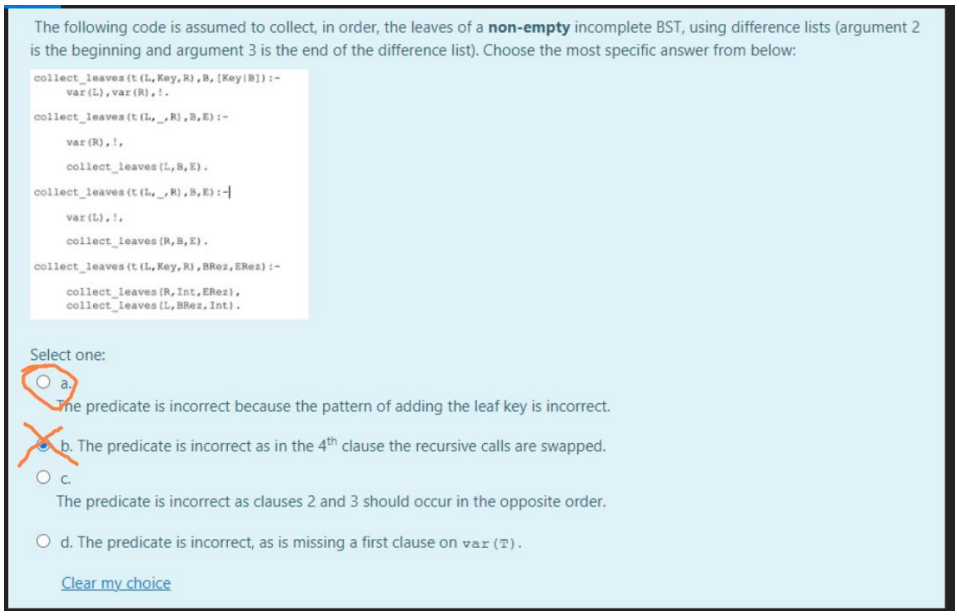
The following predicate should search for the first occurrence of an element in an *incomplete list*. It:

```
member(X, L):- var(L),fail.
member(X, [X|_]):-!.
member(X, [_|T]):-member(X,T).
```

Select one:

- a. is not correct, because it never finds the searched element
- b. is not correct, because it is deterministic
- c. is not correct, because backtracking is allowed when reaching the variable at the end of the list
- d. is correct, because the condition for reaching the end of the list is the first, and contains definitive failure

The following code is assumed to collect in order the leaves of non-empty BST…



The following code is assumed to collect, in order, the leaves of a **non-empty** incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

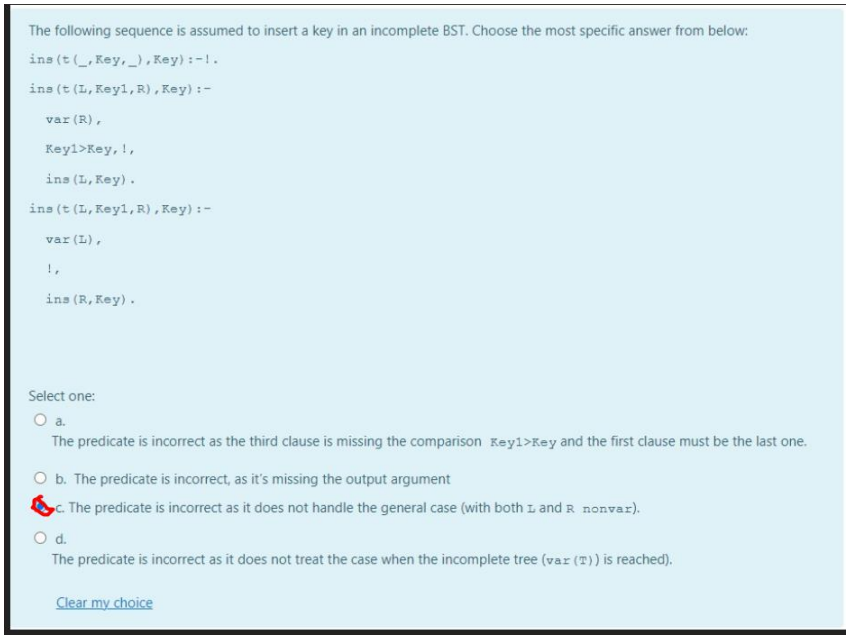
```
collect_leaves(t(L,Key,R),B,[Key|B]):-
    var(L),var(R),!.
collect_leaves(t(L,_,R),B,E):-
    var(R),!,
    collect_leaves(L,B,E).
collect_leaves(t(L,_,R),B,E):-
    var(L),!,
    collect_leaves(R,B,E).
collect_leaves(t(L,Key,R),BRez,ERez):-
    collect_leaves(R,Int,ERez),
    collect_leaves(L,BRez,Int).
```

Select one:

- a. The predicate is incorrect because the pattern of adding the leaf key is incorrect.
- b. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- c. The predicate is incorrect as clauses 2 and 3 should occur in the opposite order.
- d. The predicate is incorrect, as is missing a first clause on `var(T)`.

Clear my choice

The following sequence is assumed to insert a key in a incomplete BST……



The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

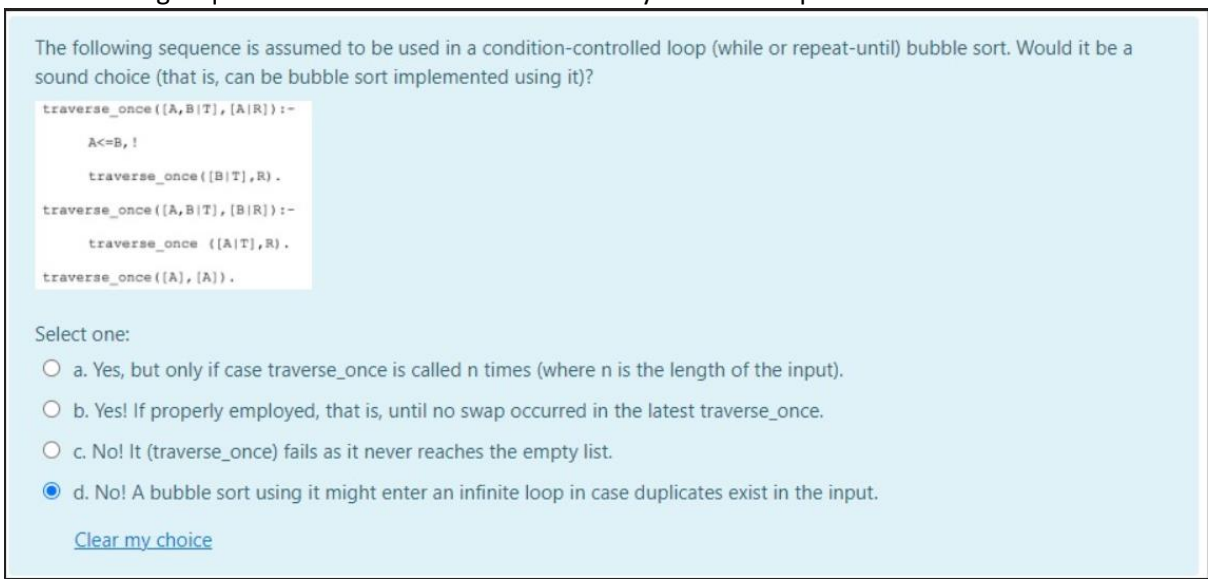
```
ins(t(_,Key,_),Key):-!.
ins(t(L,Key1,R),Key):-
  var(R),
  Key1>Key,!,
  ins(L,Key).
ins(t(L,Key1,R),Key):-
  var(L),
  !,
  ins(R,Key).
```

Select one:

- a. The predicate is incorrect as the third clause is missing the comparison `Key1>Key` and the first clause must be the last one.
- b. The predicate is incorrect, as it's missing the output argument
- c. The predicate is incorrect as it does not handle the general case (with both `L` and `R nonvar`).
- d. The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached).

Clear my choice

The Following sequence is assumed to search for a key in an incomplete BST



The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort. Would it be a sound choice (that is, can be bubble sort implemented using it)?

```
traverse_once([A,B|T],[A|R]):-
      A<=B,!
      traverse_once([B|T],R).
traverse_once([A,B|T],[B|R]):-
      traverse_once ([A|T],R).
traverse_once([A],[A]).
```

Select one:

- a. Yes, but only if case traverse_once is called n times (where n is the length of the input).
- b. Yes! If properly employed, that is, until no swap occurred in the latest traverse_once.
- c. No! It (traverse_once) fails as it never reaches the empty list.
- d. No! A bubble sort using it might enter an infinite loop in case duplicates exist in the input.

Clear my choice

The following sequence is assumed to be used in a condition controlled loop ……

The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort. Would it be a sound choice (that is, can be bubble sort implemented using it)?

```
traverse_once([A,B|T],[A|R]):-
      A<=B,!
      traverse_once([B|T],R).
traverse_once([A,B|T],[B|R]):-
      traverse_once ([A|T],R).
traverse_once([A],[A]).
```

Select one:

- a. Yes, but only if case traverse_once is called n times (where n is the length of the input).
- b. Yes! If properly employed, that is, until no swap occurred in the latest traverse_once.
- c. No! It (traverse_once) fails as it never reaches the empty list.
- d. No! A bubble sort using it might enter an infinite loop in case duplicates exist in the input.

Clear my choice

If the code below uses append with the fact as the first clause…

If the code below uses append with the fact as the first clause, for the input [4,5,3,7,4] what would be the second selection of A and B for swapping (setting B,A) during execution?

```
bubbleSort(In,Out):-
      append(X,[A,B|Y],In),
      A>B,!,
      append(X,[B,A|Y]),Int),
      bubbleSort(Int,Out).
bubbleSort(In,In).
```

Select one:

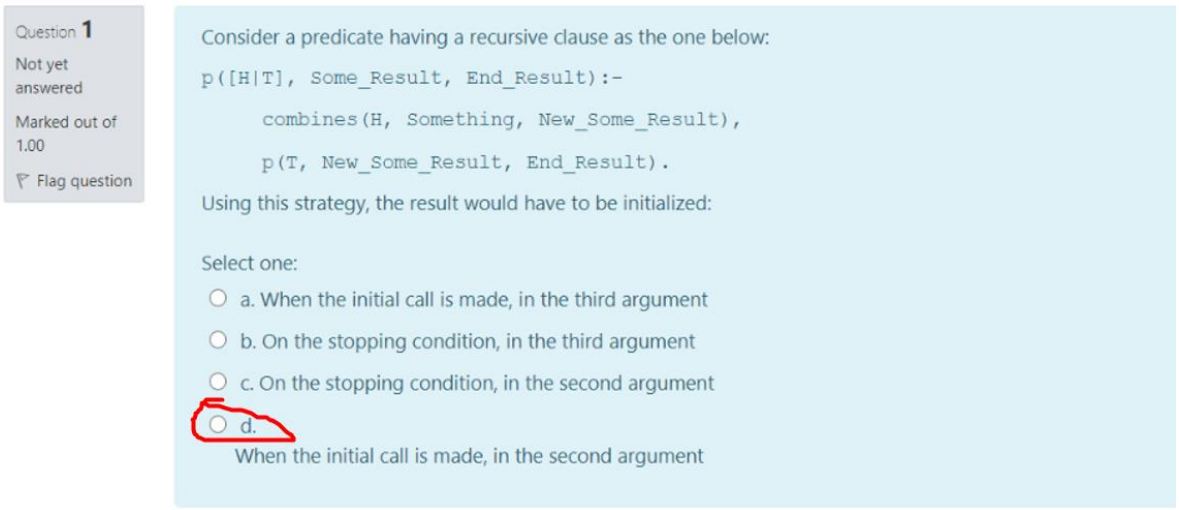
- a. A=5, B=4
- b. A=5, B=3
- c. A=4, B=3
- d. A=3, B=4

The predicate bellow should decreasingly sort by selection , using min and delete correct predicates .

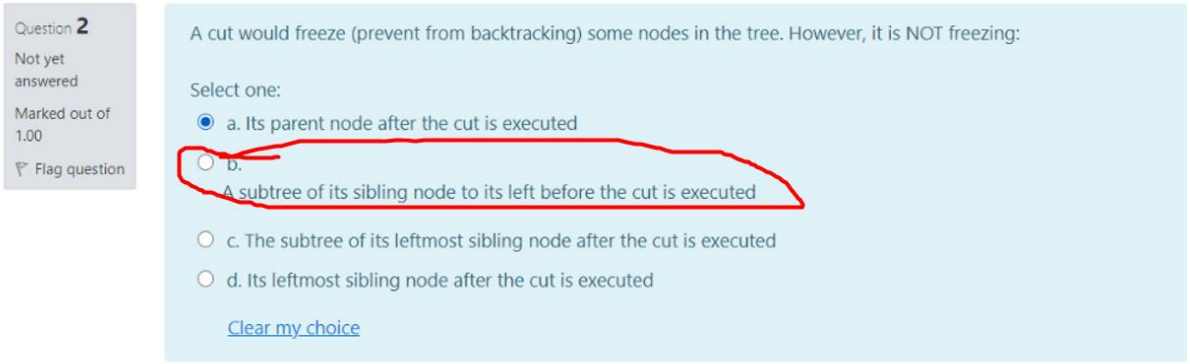
The predicate below should **decreasingly** sort by selection, using min and delete correct predicates to find the min value, and delete a value from a list, respectively. Fill in the blanks for a correct predicate:

```
wrapper(In,Out):-
   sort(In,[ ],Out).

sort(In,Acc,Out):-
      min(In,M),
      delete(M,In,Int),
      NA = [M | Acc],
      sort(Int,NA,Out).
sort([],Out,Out).
```

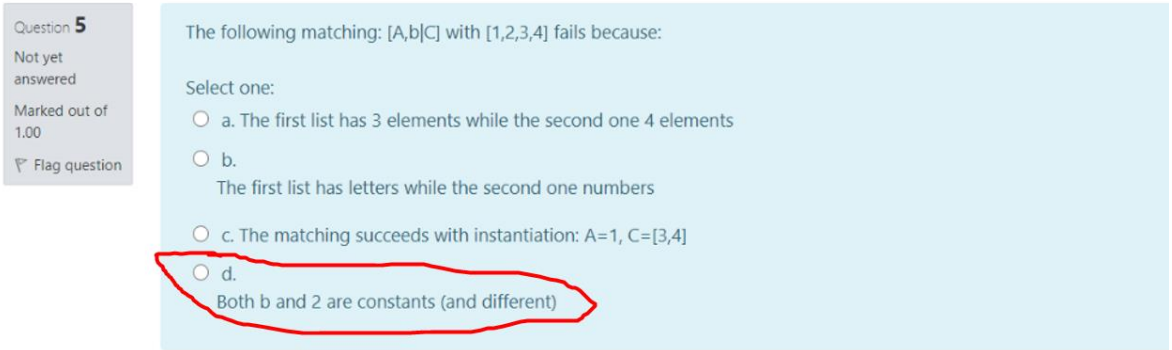
Consider a predicate having a recursive clause….



A cut would freeze (prevent from backtracking) some nodes in the tree



The following :[A,b|C] with [1,2,3,4] fails …



Which of the answers from below is correct and the most specific….

Question 1
Not yet answered
Marked out of 1.00
Flag question

Which of the answers from below is correct and the most specific?

The list `[A,_]` may consist of ...

Select one:

- a. exactly 2 elements
- b. exactly 1 element
- c. at least 1 element
- d. at least 2 elements

In search operation in a incomplete structure the following condition must hold true:

Question 2
Not yet answered
Marked out of 1.00
Flag question

In a search operation in an incomplete structure the following condition must hold true:

Select one:

- a. have a clause to check when reached a final variable: `var(V),fail,!.`
- b. have a clause to check when reached a final variable: `var(V),!,fail.`
- c. have a clause to check when reached a final variable: `fail,!,var(V).`
- d. have a clause to check when reached a final variable: `!,var(V),fail.`

Which of the following assert sequences CANNOT generate the given clause order for the dynamic

Question 3
Not yet answered
Marked out of 1.00
Flag question

Which of the following assert sequences **CANNOT** generate the given clause order for the dynamic predicate `insect/1`:

```
insect(ant).
insect(bee).
insect(moth).
insect(fly).
insect(mosquito).
```

Select one:

- a. **assert(insect(moth)) -> asserta(insect(bee)) -> asserta(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))**
- b. **asserta(insect(bee)) -> assert(insect(moth)) -> asserta(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))**
- c. **asserta(insect(moth)) -> asserta(insect(bee)) -> assertz(insect(fly)) -> assert(insect(mosquito)) -> asserta(insect(ant))**
- d. **assert(insect(mosquito)) -> asserta(insect(fly)) -> assert(insect(moth)) -> asserta(insect(bee)) -> asserta(insect(ant))**

I want to write a predicate which computes the minimum key in a BST . Complete the partial

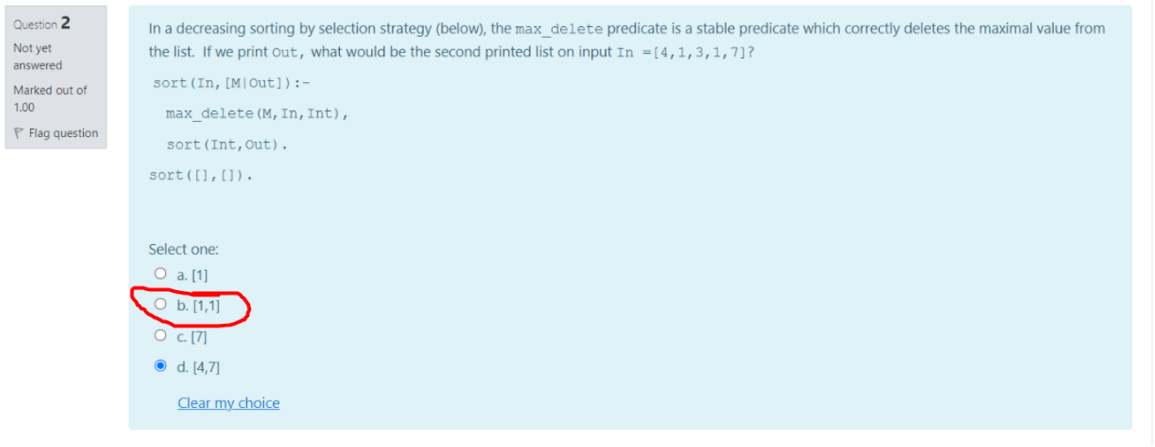
Question 4
Not yet answered
Marked out of 1.00
Flag question

I want to write a predicate which computes the minimum key in a BST. Complete the partial implementation so that you get a correct predicate:

min(t(K, Nil, _), K).

min(t(K,L,R), M):-min(L , M).

In a decreasing sorting by selection strategy (below) the max_delete



Question 2
Not yet answered
Marked out of 1.00
Flag question

In a decreasing sorting by selection strategy (below), the `max_delete` predicate is a stable predicate which correctly deletes the maximal value from the list. If we print `Out`, what would be the second printed list on input `In =[4,1,3,1,7]`?

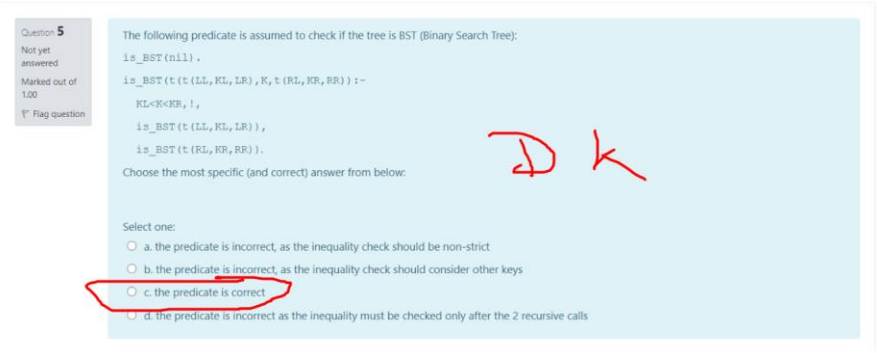
```
sort(In,[M|Out]):-
  max_delete(M,In,Int),
  sort(Int,Out).
sort([],[]).
```

Select one:

- a. [1]
- b. [1,1]
- c. [7]
- d. [4,7]

Clear my choice

The following predicate is assumed to check if the tree ….



Question 5
Not yet answered
Marked out of 1.00
Flag question

The following predicate is assumed to check if the tree is BST (Binary Search Tree):

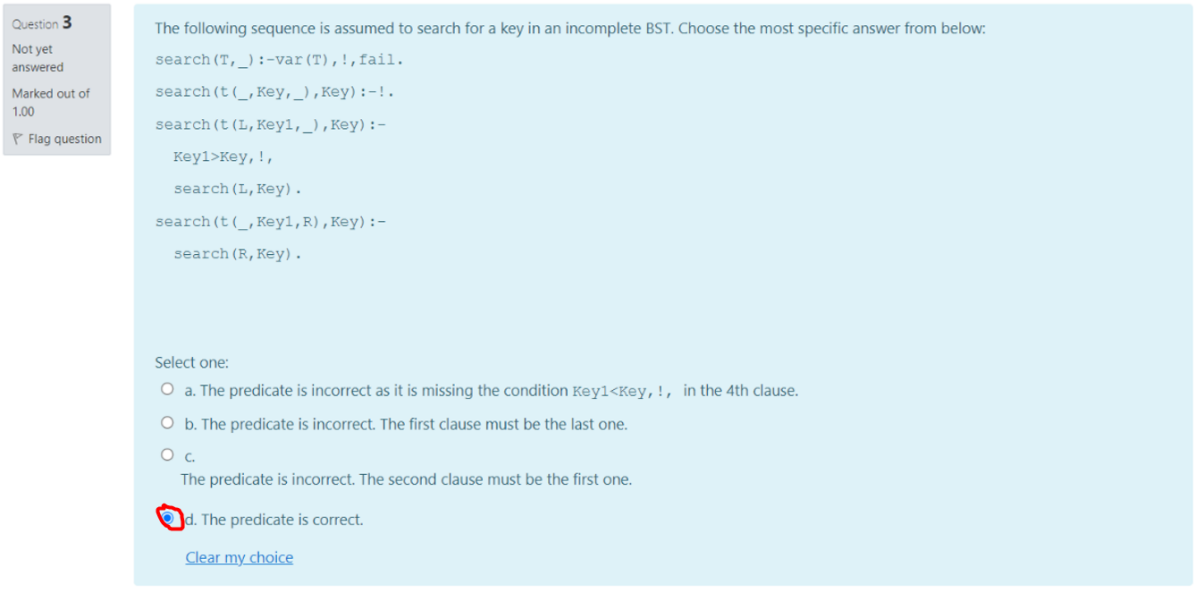
```
is_BST(nil).
is_BST(t(t(LL,KL,LR),K,t(RL,KR,RR)):-
  KL<K<KR,!,
  is_BST(t(LL,KL,LR)),
  is_BST(t(RL,KR,RR)).
```

Choose the most specific (and correct) answer from below:

Select one:

- a. the predicate is incorrect, as the inequality check should be non-strict
- b. the predicate is incorrect, as the inequality check should consider other keys
- c. the predicate is correct
- d. the predicate is incorrect as the inequality must be checked only after the 2 recursive calls

The following sequence is assumed to search for a key in a incomplete BST. Choose the most specific



Question 3
Not yet answered
Marked out of 1.00
Flag question

The following sequence is assumed to search for a key in an incomplete BST. Choose the most specific answer from below:

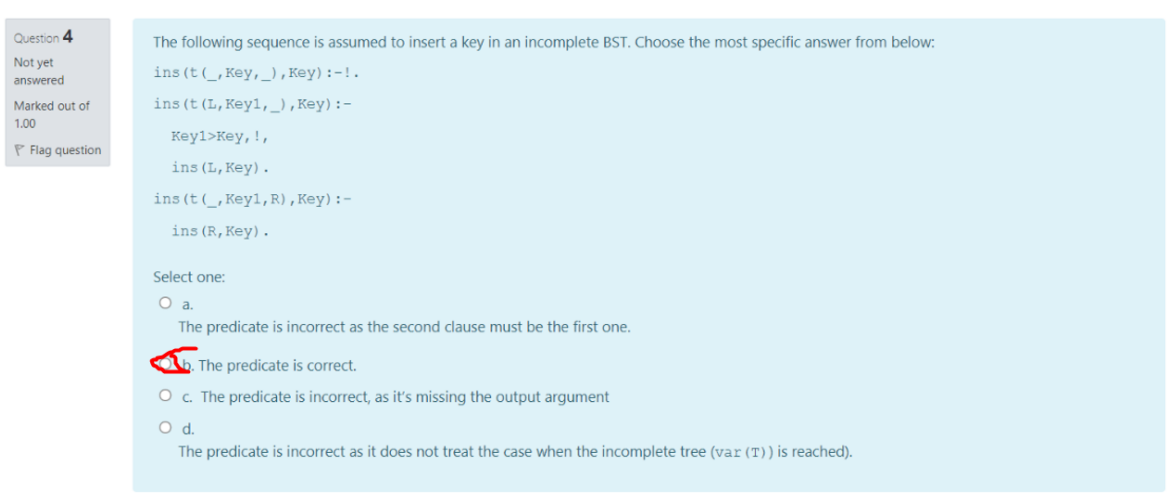
```
search(T,_):-var(T),!,fail.
search(t(_,Key,_),Key):-!.
search(t(L,Key1,_),Key):-
  Key1>Key,!,
  search(L,Key).
search(t(_,Key1,R),Key):-
  search(R,Key).
```

Select one:

- a. The predicate is incorrect as it is missing the condition `Key1<Key,!,` in the 4th clause.
- b. The predicate is incorrect. The first clause must be the last one.
- c. The predicate is incorrect. The second clause must be the first one.
- d. The predicate is correct.

Clear my choice

The following sequence is assumed to inset a ley in an incomplete BST . Choose the most specfi



Question 4

Not yet answered

Marked out of 1.00

Flag question

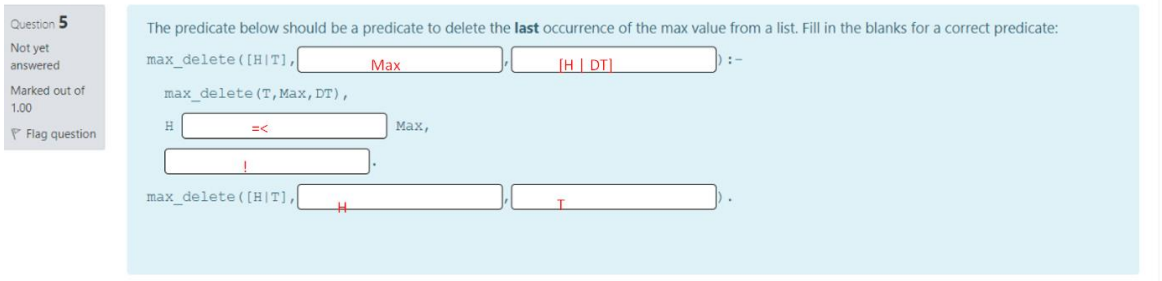
The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```
ins(t(_,Key,_),Key):-!.
ins(t(L,Key1,_),Key):-
  Key1>Key,!,
  ins(L,Key).
ins(t(_,Key1,R),Key):-
  ins(R,Key).
```

Select one:

- a. The predicate is incorrect as the second clause must be the first one.
- b. The predicate is correct.
- c. The predicate is incorrect, as it's missing the output argument
- d. The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached).

The predicate below should be a predicate to delete the last occurrence….



Question 5

Not yet answered

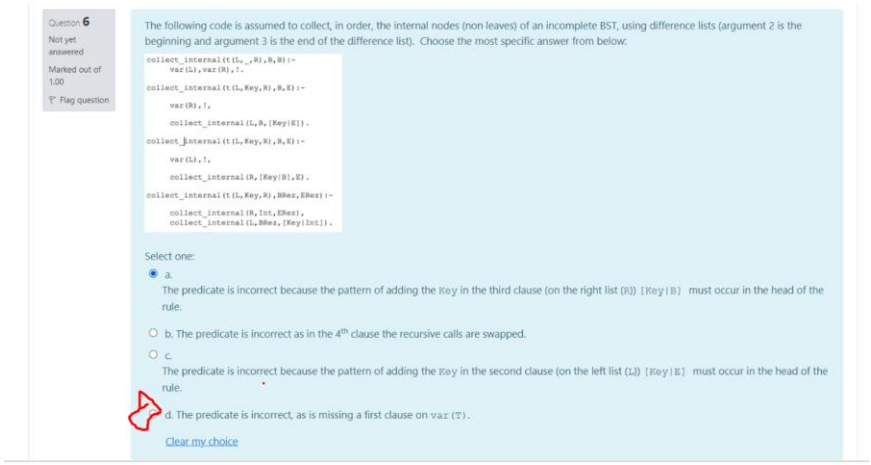
Marked out of 1.00

Flag question

The predicate below should be a predicate to delete the **last** occurrence of the max value from a list. Fill in the blanks for a correct predicate:

```
max_delete([H|T], Max , [H | DT] ):-
  max_delete(T,Max,DT),
  H  =<  Max,
  !.
max_delete([H|T], H , T ).
```

The following code is assumed to collect in order the internal nodes (non leaves(of an incomplete BST) (nu e sigur)



Question 6

Not yet answered

Marked out of 1.00

Flag question

The following code is assumed to collect, in order, the internal nodes (non leaves) of an incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```
collect_internal(t(L,_,R),B,B):-
    var(L),var(R),!.
collect_internal(t(L,Key,R),B,E):-
    var(R),!,
    collect_internal(L,B,[Key|E]).
collect_internal(t(L,Key,R),B,E):-
    var(L),!,
    collect_internal(R,[Key|B],E).
collect_internal(t(L,Key,R),BRez,ERez):-
    collect_internal(R,Int,ERez),
    collect_internal(L,BRez,[Key|Int]).
```

Select one:

- a. The predicate is incorrect because the pattern of adding the `Key` in the third clause (on the right list (`R`)) `[Key|B]` must occur in the head of the rule.
- b. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- c. The predicate is incorrect because the pattern of adding the `Key` in the second clause (on the left list (`L`)) `[Key|E]` must occur in the head of the rule.
- d. The predicate is incorrect, as is missing a first clause on `var(T)`.

Clear my choice

The following code is assumed to count the internal nodes with 2 children of an incomplete BT.

Question **7**

Not yet answered

Marked out of 1.00

Flag question

The following code is assumed to count the internal nodes with 2 children of an incomplete binary tree. Choose the most specific answer from below:

```
count_2_children(t(L,_,R),N):-var(L),!,
    count_2_children(R,N).
count_2_children(t(L,_,R),N):-var(R),!,
    count_2_children(L,N).
count_2_children(t(L,_,R),NN):-
    count_2_children(R,NR),
    count_2_children(L,NL),
    NN is NL+NR+1.
count_2_children(t(L,_,R),0):-
    var(L),var(R),!.
```

Select one:

- a. The predicate is incorrect, as is missing a clause on `var(T)`.
- b. The predicate is incorrect as the last clause should be the first one.
- c. The predicate is correct.
- d. The predicate is incorrect as the clause right subtree is processed before the left one.

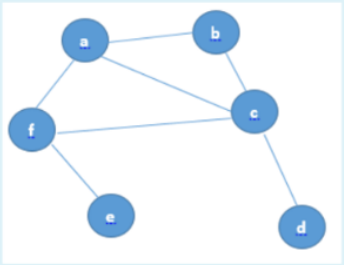
You are given the predicate below and the example graph on the right blab la bla

You are given the predicate below and the example graph on the right (undirected, *edge-clause* form, nodes appear in lexicographic order):

```
d(X):- asserta(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).
```

What will the predicate base look like after the execution of:

?- d(a).



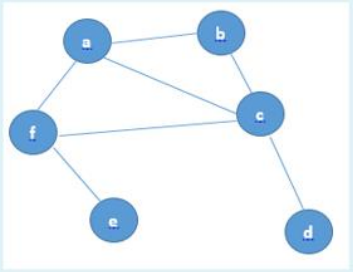
Select one or more:

- [ ] a. vert(a).
  vert(b).
  vert(c).
  vert(d).
- [ ] b. vert(a).
  vert(b).
  vert(c).
  vert(d).
  vert(f).
  vert(e).
- [x] c. vert(e).
  vert(f).
  vert(d).
  vert(c).
  vert(b).
  vert(a).
- [ ] d. vert(d).
  vert(c).
  vert(b).

retry(a).

You are given the predicate below and the example graph on the right (undirected, *edge-clause* form, nodes appear in lexicographic order):

```
d(X):- assertz(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).
d(_).
```



What will the predicate base look like after the execution of:

?- d(a).

Select one or more:

- [ ] a. vert(a). vert(b). vert(c). vert(d). vert(e). vert(f).
- [x] b. vert(e). vert(f). vert(d). vert(c). vert(b). vert(a).
- [ ] c. vert(a). vert(b). vert(c). vert(d).
- [x] d. vert(d). vert(c). vert(b). vert(a).

The following code is assumed to calculate the internal nodes with 2 children of BT bla bla

Question **2**

Not yet answered

Marked out of 1.00

Flag question

The following code is assumed to calculate the internal nodes with 2 children of a binary tree. Choose the most specific answer from below:

```
count_2_children(t(L,_,R),NN):-
    count_2_children(L,NL),
    count_2_children(R,NR),
    NN is NL+NR.
count_2_children(t(nil,_,R),N):-
    count_2_children(R,N).
count_2_children(t(L,_,nil),N):-
    count_2_children(L,N).
count_2_children(t(nil,_,nil),0).
```

Select one:

- a. The predicate is incorrect as the clause with 2 recursive calls is not counting the node
- b. The predicate is incorrect as the last clause should be the first one.
- c. The predicate is incorrect, as is missing a clause on `nil`.
- d. The predicate is correct.

The following code is assumed to collect in order the leaves of a non empty bla blaS

Question **3**

Not yet answered

Marked out of 1.00

Flag question

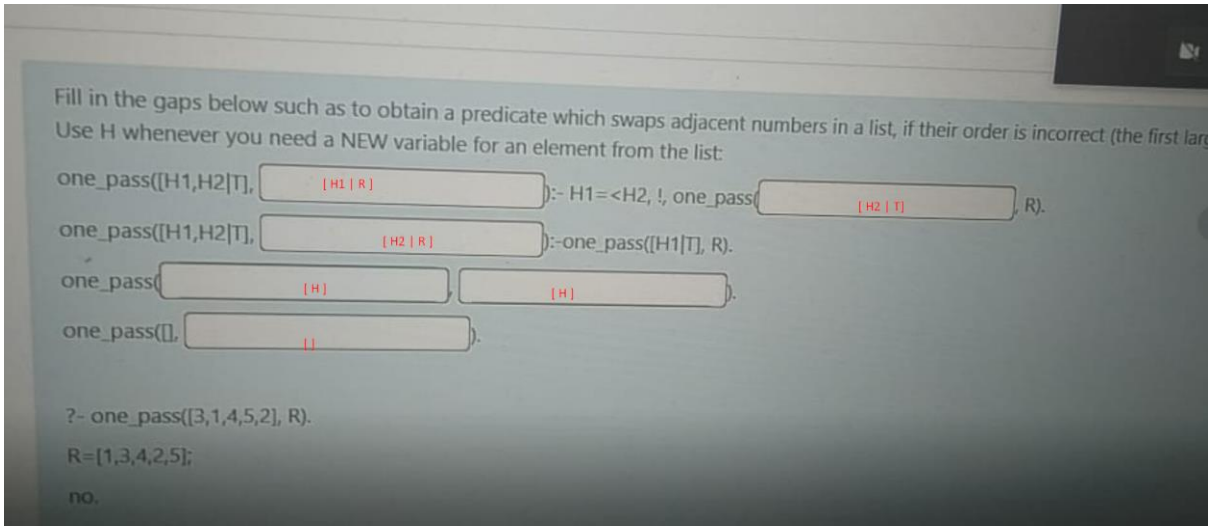
The following code is assumed to collect, in order, the leaves of a **non-empty** incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```
collect_leaves(t(L,Key,R),B,[Key|B]):-
    var(L),var(R),!.
collect_leaves(t(L,_,R),B,E):-
    var(R),!,
    collect_leaves(L,B,E).
collect_leaves(t(L,_,R),B,E):-
    var(L),!,
    collect_leaves(R,B,E).
collect_leaves(t(L,Key,R),BRez,ERez):-
    collect_leaves(R,Int,ERez),
    collect_leaves(L,BRez,Int).
```

Select one:

- a. The predicate is incorrect, as is missing a first clause on `var(T)`.
- b. The predicate is incorrect as clauses 2 and 3 should occur in the opposite order.
- c. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- d. The predicate is incorrect because the pattern of adding the leaf key is incorrect.

Fil in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list



1 . Complete the following predicate to make…

Complete the following predicate to make it find cycles in a graph containing the node X:

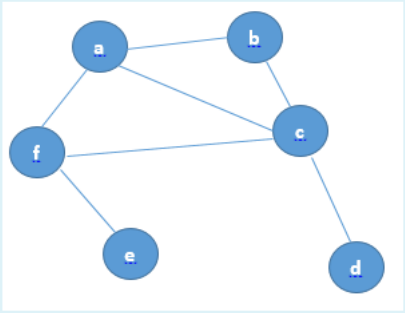
```
c(X, [Y |P]):- edge(X,Y), c(Y,X,[Y],P).

c(X,Y,PP, FP):-
    edge(X, Z),
    \+(member(Y, PP)),
    c(X, Z, [Z|PP], FP).
c(X, Y, PP, P):- reverse(PP, P).
```

You are given the predicate below and the example graph on the right (undirected, *edge-clause* form, nodes appear in lexicographic order):

```
d(X):- assertz(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).
d(_).
```

What will the predicate base look like after the execution of:

?- d(a).



Select one or more:

- [ ] a. vert(a).
  vert(b).
  vert(c).
  vert(d).
  vert(e).
  vert(f).
- [x] b. vert(e).
  vert(f).
  vert(d).
  vert(c).
  vert(b).
  vert(a).
- [ ] c. vert(a).
  vert(b).
  vert(c).

  vert(d).
- [x] d. vert(d).
  vert(c).
  vert(b).
  vert(a).

The following code is assumed to collect, in order, the leaves of a **non-empty** incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```
collect_leaves(t(L,Key,R),B,[Key|B]):-
    var(L),var(R),!.

collect_leaves(t(L,_,R),B,E):-

    var(R),!,

    collect_leaves(L,B,E).

collect_leaves(t(L,_,R),B,E):-

    var(L),!,

    collect_leaves(R,B,E).

collect_leaves(t(L,Key,R),BRez,ERez):-

    collect_leaves(R,Int,ERez),
    collect_leaves(L,BRez,Int).
```

Select one:

- ( ) a.
  The predicate is incorrect because the pattern of adding the leaf key is incorrect.
- (•) b. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- ( ) c.
  The predicate is incorrect as clauses 2 and 3 should occur in the opposite order.
- ( ) d. The predicate is incorrect, as is missing a first clause on `var(T).`

Clear my choice

The following code is assumed to count the internal nodes with 2 children of an incomplete binary tree. Choose the most specific answer from below:

```
count_2_children(t(L,_,R),N):-var(L),!,
     count_2_children(R,N).
count_2_children(t(L,_,R),N):-var(R),!,
     count_2_children(L,N).
count_2_children(t(L,_,R),NN):-
     count_2_children(R,NR),
     count_2_children(L,NL),
     NN is NL+NR+1.
count_2_children(t(L,_,R),0):-
     var(L),var(R),!.
```

Select one:

- a. The predicate is incorrect as the clause right subtree is processed before the left one.
- b. The predicate is incorrect as the last clause should be the first one.
- c. The predicate is correct.
- d. The predicate is incorrect, as is missing a clause on `var(T)`.

Clear my choice

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```
ins(t(L,Key1,_),Key):-
  Key1>Key,!,
  ins(L,Key).
ins(t(_,Key1,R),Key):-
  ins(R,Key).
ins(t(_,Key,_),Key):-!.
```

Select one:

- a. The predicate is incorrect as the last clause must be the first one.
- b. The predicate is incorrect, as it's missing the output argument
- c. The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached).
- d. The predicate is incorrect as the second clause is missing the comparison Key1<Key.

Clear my choice